

Unveiler: Modular Spatial Reasoning for Sequential Manipulation in Dense Clutter

Chrisantus Eze^{1*}, Ryan C Julian² and Christopher Crick¹
Oklahoma State University¹, Google DeepMind²

Abstract—We argue that explicit compositional decomposition is the right inductive bias for sequential manipulation in dense clutter. By separating long-horizon spatial reasoning from low-level action execution into independently trainable modules, each component learns a strictly simpler problem, failures become independently diagnosable, and the system remains deployable on resource-constrained hardware. We present Unveiler, which instantiates this principle via two modules: a lightweight transformer-based Spatial Relationship Encoder (SRE) that sequentially identifies the optimal obstacle for removal, and a rotation-invariant Action Decoder that executes the physical removal. The modules communicate through a clean discrete interface (a single object index), enabling the SRE to transfer zero-shot to real scenes without any fine-tuning, while the Action Decoder requires only a one-time geometric recalibration of workspace bounds. The SRE is trained in two stages: imitation learning from heuristic demonstrations for sample-efficient initialization, followed by PPO fine-tuning that enables the policy to discover removal strategies surpassing the heuristic in dense clutter, exploiting the modular structure to focus exploration where the heuristic degrades. At 83.03M parameters and ~ 260 ms inference, Unveiler achieves up to 97.6% success in partially occluded and 90.0% in fully occluded scenarios, outperforming monolithic and VLA baselines by 15–40%. Our results provide an empirical existence proof that compositional decomposition yields systems that are simultaneously more accurate, more efficient, more interpretable, and more transferable than monolithic alternatives on this class of sequential manipulation tasks.

I. INTRODUCTION

Sequential manipulation in dense clutter is a natural test case for compositional robotics. The task decomposes cleanly into two functionally distinct sub-problems: *which* objects to remove and in *what order* (long-horizon spatial reasoning), and *how* physically to remove each one (low-level action execution). Methods that conflate these into a single monolithic mapping from pixels to grasp actions [1]–[4] force one model to solve both jointly, which scales poorly as scene complexity grows. Large-scale vision-language-action (VLA) approaches [5]–[7] offer impressive generality but at either prohibitive computational cost or long inference latencies exceeding 6,000ms per decision (see Table I), neither of which fits the tight budgets of real manipulation systems.

We argue that *explicit compositional decomposition* is the right inductive bias here. By separating a lightweight spatial reasoning module (which identifies the next obstacle) from a dedicated execution module (which removes it), each

component learns a strictly simpler problem. Crucially, the two modules communicate through a clean discrete interface (a single object index), enabling independent training, independent failure diagnosis, and independent transfer. Figure 1 shows this modular pipeline executing on a real robot with no retraining: the SRE selects obstacles, the Action Decoder removes them, and the target is successfully grasped.

The Unveiler’s Spatial Relationship Encoder (SRE) and Action Decoder utilize only 83.03M parameters yet achieve 97.6% success in partially occluded scenarios, outperforming models with significantly larger parameter counts [8]–[10] by over 20% (see Tables I and II). Furthermore, we show that VLAs can reason spatially in simple scenes but degrade in dense clutter, whereas the modular SRE transfers zero-shot to real images without fine-tuning.

Our primary contributions are:

- **A compositionally decomposed architecture** that separates spatial reasoning from action execution via a discrete object-index interface. Independent training of the two modules yields 15–40% higher success rates than monolithic approaches across varying clutter densities, while keeping each module independently diagnosable and transferable.
- **A transformer-based Spatial Relationship Encoder (SRE)** that learns to identify optimal obstacle removal sequences from heuristic demonstrations, achieving up to 97.6% accuracy in densely cluttered scenes. The SRE transfers zero-shot to real top-down images without any fine-tuning, while the Action Decoder requires only a one-time geometric recalibration.
- **A modular two-stage training pipeline** in which imitation learning initializes the SRE sample-efficiently, and PPO fine-tuning then exploits the modular structure to explore beyond the heuristic in the dense, fully occluded regime where it degrades.

We train our framework entirely in PyBullet simulation [11] and evaluate it across scene complexities of 2–12 objects, varying occlusion levels, and both simulated and real-world settings. Our results make a case for the continued relevance of purpose-built compositional architectures in an era of large-scale models: they can serve as lightweight, interpretable, and deployable components that complement or specialize foundation models for structured manipulation tasks.

*Corresponding author

²This work was done when Ryan C. Julian was at Google DeepMind

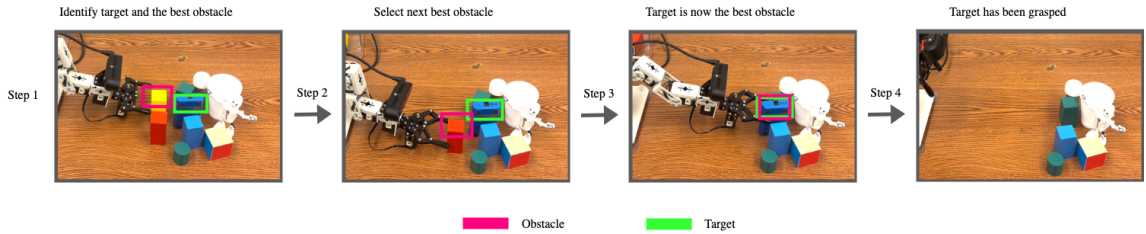


Fig. 1. Unveiler executing on a real robot (Dofbot-Pro). The SRE module selects the pink-bounded obstacle at Step 1 and Step 2, sequentially removing it to expose the green-bounded target. By Step 3 the target is accessible; Step 4 shows the successful grasp. This step-by-step execution illustrates the modular decomposition in action: the SRE reasons about object ordering independently of the Action Decoder, which handles physical removal. No retraining was performed for real-world deployment.

II. RELATED WORK

A. Push-Grasping Synergy

Coordinating pushing and grasping to manage clutter is well-studied [1], [12]–[14]. TransporterNet [2] uses a fully convolutional network for push-grasp visuomotor policies. More recent methods [3], [4] integrate “move”, “push”, and “grasp” actions for target-oriented grasping. Unlike these methods, which focus on clearing clutter, Unveiler reasons about spatial relationships between objects to determine an optimal removal sequence via a dedicated reasoning module, rather than learning the two tasks jointly.

B. Object-centric Representations and VLMs

Object-centric world models for manipulation [15]–[18] improve generalization by encoding per-object structure. Unveiler specializes this toward sequential clutter: rather than general generalization, the SRE explicitly encodes long-horizon spatial dependencies and occlusion relationships, yielding interpretable decision-making via inspectable selection probabilities.

VLM-based approaches [8], [10], [19], [20] leverage semantic understanding for flexible grasping but carry significant computational overhead and struggle with the fine-grained spatial reasoning needed in dense clutter (see Table I). Unveiler complements these: the SRE can serve as a lightweight spatial reasoning module that interfaces with a VLA’s execution layer, an instance of modularity enabling the best of both worlds.

C. Compositional and Modular Approaches in Robotics

Compositional or modular learning exploits problem structure by building abstractions between functionally distinct components, reducing sample complexity and improving generalization [21]. In robotic manipulation, this principle manifests in task-and-motion planning (TAMP) [22], where geometric reasoning and symbolic planning are decoupled; in option frameworks that compose reusable skills for long-horizon tasks; and in hierarchical imitation learning that separates high-level goal selection from low-level execution.

Recent work has revisited compositionality in the era of foundation models, noting that scaling alone does not easily address data-scarce modalities (e.g., force and tactile sensing) or the compositional structure of dexterous human actions. A parallel trend in VLA research similarly

advocates decoupling high-level reasoning from low-level control, using visual prompting or language-conditioned sub-policies as the execution layer; Unveiler pursues the same principle but instantiates it with a lightweight, object-indexed interface specialized for dense-clutter retrieval, avoiding the large pretraining footprint that VLA-based decompositions typically require, while retaining the core benefits: independent training, independent transfer, and independent failure diagnosis.

III. PROBLEM FORMULATION AND DECOMPOSITION

We study sequential target retrieval from dense clutter using a top-down RGB-D perception stack and a push-grasp primitive. Given an RGB scene image I and corresponding depth heightmap I_g (constructed from the fused RGB-D point cloud), instance segmentation produces a set of object masks $M = \{M_1, \dots, M_N\}$ and a target index t . We use heightmaps rather than raw depth images because they provide a robot-centric, top-down representation that reduces perspective distortion and yields a consistent action reference frame.

Segmentation mismatch. In clutter, the number of detected instances may differ from the true object count. We denote by N the number of predicted masks at the current step; Unveiler predicts and executes one action per step, so each removal typically improves visibility and reduces segmentation errors over time.

Action primitive and constraints. Following Kiatos et al. [3], the robot executes a push-grasp primitive under the constraints: (i) fixed end-effector height during execution, (ii) constant aperture during the push phase, (iii) heightmap boundaries aligned with workspace limits, and (iv) a $0.5\text{ m} \times 0.5\text{ m}$ tabletop workspace.

At each decision step, the robot must (a) choose which visible object to remove next and (b) execute a continuous push-grasp action to remove it. Unveiler factorizes this into a discrete object-selection policy (SRE) and a conditional action-execution policy (Action Decoder):

$$\pi(a | s) = \sum_{i \in \{1, \dots, N\}} \pi_{\text{SRE}}(i | s, t) \pi_{\text{AD}}(a | s, i), \quad (1)$$

where the state s comprises the scene observation (including I and/or I_g) and the set of instance masks/crops M with target index t , the discrete choice i indexes an object instance, and $a \in \mathbb{R}^4$ parameterizes the push-grasp primitive (position,

orientation, aperture). This decomposition enables the SRE to focus on object-centric spatial reasoning (which obstacle is most critical), while the Action Decoder focuses on robust, rotation-invariant execution for the selected object.

This factorization reduces the learning problem to multiclass object selection and yields an interpretable bound relating end-task performance to SRE selection error (Section V).

IV. PROPOSED APPROACH

Our framework determines the next object to remove based on two spatial criteria: (i) distance to the workspace boundary, which proxies grasp accessibility (peripheral objects are less occluded and more reachable); and (ii) distance from the target, prioritizing objects in closest proximity to the obstructed region. By removing peripheral objects first, scene stability is promoted implicitly. We decompose this into two complementary components: a Spatial Relationship Encoder (Section IV-A) that learns to identify the optimal next obstacle, and an Action Decoder (Section IV-B) that executes the physical removal.

A. Spatial Relationship Encoder

While the heuristic (Algorithm 1) provides effective demonstrations in moderate clutter, it operates on geometric primitives (centroid positions and boundary distances) that become unreliable under dense occlusion and imperfect segmentation. The SRE is designed to internalize the same spatial logic into learned visual representations, gaining robustness to segmentation noise and the ability to capture higher-order object dependencies that the heuristic’s additive scoring cannot model.

The SRE is a transformer encoder model [23] and takes as input the top-camera view image of the scene, a cropped view of the target object, and the cropped views of all visible objects in the scene. The architecture is designed to output a discrete decision: the index of the object that should be removed next. These indices correspond directly to the N object bounding boxes detected by the segmentation model, where each detected object is assigned a sequential index from 0 to $N-1$. The SRE then outputs a probability distribution over these N indices, selecting the object with the highest removal probability. Invalid indices (those exceeding the current object count) are automatically filtered out during inference.

The inputs are normalized and replicated across three channels to match the expected input format. Visual features are then extracted using a ResNet18 [24] backbone pretrained on ImageNet [25], transforming each $1 \times 144 \times 144$ image into a 1×1024 feature map for both the target and the N object crops. These features are flattened and passed through multi-layer perceptrons (MLPs) to obtain 1024-dimensional latent representations. A multi-head attention mechanism computes attention scores between the target (as query), scene image (as key), and the surrounding objects (as value). An MLP further processes the resulting $N \times 512$ attention output to produce $1 \times N$ removal weights for each object.

To model inter-object dependencies and target relevance, we employ a stack of transformer blocks. These layers enable the model to capture higher-order relationships such as indirect occlusions (i.e., an object indirectly blocking the target by being in front of another obstacle).

The output of this module is a selection mask highlighting which object should be passed to the Action Decoder for manipulation. This discrete selection mechanism enables sequential decision making, where only one object is removed per step, allowing the robot to reduce occlusion and scene clutter iteratively.

SRE Training: The SRE is trained in two stages. In the first stage, the policy is initialized via imitation learning (IL) on demonstrations generated by the scoring heuristic in Algorithm 1. This encodes the geometric removal logic (peripheral accessibility and target proximity) into learned visual representations, providing a strong, sample-efficient starting point that already generalizes well in moderate clutter.

In the second stage, we fine-tune the IL-pretrained weights with proximal policy optimization (PPO) [26], formulating obstacle selection as a discrete Markov Decision Process (MDP). The state s_t is the SRE’s visual input (scene heightmap, target crop, and object crops), and the action $a_t \in \{1, \dots, N\}$ is the index of the selected obstacle. Because the IL stage initiates the policy in a well-structured action space, RL exploration is directed toward the dense, fully occluded regimes where the heuristic degrades. Training runs entirely in simulation with no additional real-robot data.

Each episode samples $N \in [2, 12]$ objects uniformly, with horizon H and discount γ . The per-step reward is

$$r_t = \alpha \cdot \mathbf{1}[\text{success}] \cdot \left(1 + \beta \frac{H-t}{H}\right) + r_{\text{access}}(a_t, s_t) + r_{\text{occl}}(a_t, s_t) \cdot \mathbf{1}[a_t \neq \text{tgt}] + r_{\text{step}}, \quad (2)$$

where $r_{\text{access}} = +1$ rewards grasping the target when accessible; $r_{\text{occl}} \in \{+0.5, +0.25\}$ rewards obstacle removals that yield direct target exposure or indirect path clearance, respectively; $r_{\text{step}} = -0.1$ penalizes each unnecessary action; $\alpha = 2$, $\beta = 0.5$ scale the time-bonus; $H = N$; $\gamma = 0.99$. Direct exposure is detected when the target’s pixel-level mask area increases after the removal; indirect clearance when any object on the heuristic removal path is removed.

B. Action Decoder

Once the SRE identifies the object that should be removed, the Action Decoder generates the low-level action required to execute the removal. This module is responsible for determining a robust push-grasp action, including position and rotation, tailored to the selected object and the current scene configuration.

The Action Decoder is a fully convolutional network (FCN) that takes two inputs: a depth heightmap of the full scene I_g , and a crop of the selected object O_i extracted from the depth heightmap. These inputs are passed through separate convolutional pathways (see Figure 2). The streams

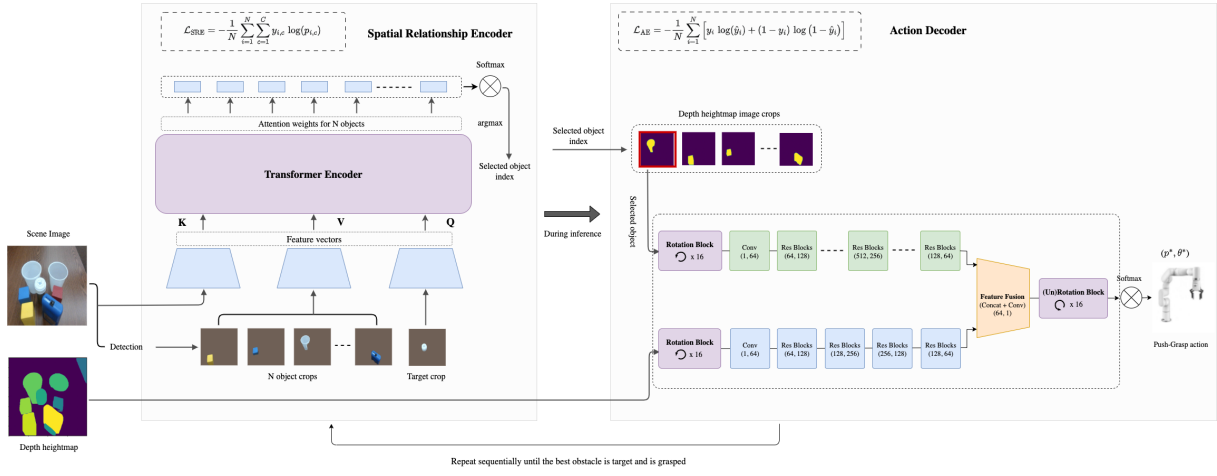


Fig. 2. Unveiler system architecture. Independent training of the SRE and Action Decoder enables each component to specialize in its respective task while sharing a discrete object index at inference. The SRE processes scene and object-centric visual inputs through a transformer to select the optimal obstacle, and the Action Decoder generates rotation-invariant push-grasp parameters for the selected object.

Algorithm 1: Obstacle Selection Heuristic

Require: Masks $M = \{M_i\}_{i=1}^N$, target index t
Ensure: Removal order R

- 1: **if** $|M| \leq 3$ **then**
- 2: **return** $[t]$
- 3: **end if**
- 4: Compute $\hat{d}_{\text{edge}}, \hat{d}_{\text{target}}$: normalized boundary and centroid distances
- 5: $s[i] \leftarrow \hat{d}_{\text{edge}}[i] + \hat{d}_{\text{target}}[i] \quad \forall i \neq t$
- 6: **return** $R \leftarrow \text{argsort}(s)$

are fused to produce grasp heatmaps across 16 discrete orientations (22.5° increments) for rotation invariance.

The Action Decoder is trained using a supervised cross-entropy loss over grasp maps. During evaluation, we assess performance via metrics such as grasp success rate and number of actions (steps) required to retrieve the target. The expert used to obtain the supervision label is shown in Algorithm 2.

C. Data Collection

Our policy is trained entirely through demonstrations collected in the PyBullet [11] simulation environment. We develop a set of heuristic functions that identify both the optimal obstacle to remove and its corresponding grasp configuration at each decision step (see Algorithms 1 and 2). We filter demonstrations to include only successful attempts, defined by single-object interaction and grasp stability (the object must remain held when lifted). Identical criteria are used during evaluation for faithful assessment.

V. PROPERTIES OF THE DECOMPOSED POLICY

The factorization equation (Eqn. 1) defined in Section III is not merely an architectural convenience. It induces two properties with direct implications for how the system learns and how failures can be diagnosed.

Algorithm 2: Grasp Pose Heuristic

Require: Depth heightmap I_g , object mask M_i , workspace bounds $[d_{\min}, d_{\max}]$
Ensure: Push-grasp action $\mathbf{a} = [p_x, p_y, \theta, a_{\text{apt}}]^\top$

- 1: $\mathcal{V} \leftarrow \{(x, y) \mid d_{\min} \leq d(x, y) \leq d_{\max}\}$ {valid grasp pixels within workspace}
- 2: **if** $\mathcal{V} = \emptyset$ **then**
- 3: **return** $\mathbf{0}$
- 4: **end if**
- 5: $\mathcal{C} \leftarrow \text{contours}(M_i)$ {object boundary pixels}
- 6: **repeat**
- 7: $p_1 \sim \mathcal{U}(\mathcal{V})$ {sample grasp center}
- 8: $\mathcal{P} \leftarrow \{p \in \mathcal{C} \mid \|p - p_1\|_\infty < d_{\text{push}}\}$ {nearby boundary points}
- 9: **until** $\mathcal{P} \neq \emptyset$
- 10: $p_2 \sim \mathcal{U}(\mathcal{P})$ {push target on object boundary}
- 11: $\theta \leftarrow \text{discretize}_{22.5^\circ}(-\arctan2(p_2 - p_1))$ {quantized push direction}
- 12: $a_{\text{apt}} \sim \mathcal{U}(\text{aperture limits})$ {gripper opening width}
- 13: **return** $\mathbf{a} = [1.05 \cdot p_1, \theta, a_{\text{apt}}]^\top$

1) *Object-centric inductive bias:* The SRE architecture encodes structure that a monolithic policy must discover implicitly. The target is always the cross-attention query, anchoring learned representations to target-relative spatial context from the first gradient update. Per-object crops make the model equivariant to instance ordering, so the same selection behavior generalizes across object counts and configurations not seen during training.

2) *Additive error decomposition:* Because selection and execution are separate, their failure modes are also separate. If the SRE selects a suboptimal object on a fraction ϵ_{SRE} of steps and each wrong choice costs at most Δ in expected progress, the total performance gap of the actual SRE policy

$J(\pi)$) to an ideal policy ($J(\pi^*)$) is bounded by

$$J(\pi^*) - J(\pi) \leq H \cdot \Delta \cdot \epsilon_{\text{SRE}} + \epsilon_{\text{exec}} \quad (3)$$

where $H \leq N$ is the episode horizon (one step per removal), Δ is the worst-case cost of a single wrong object pick, ϵ_{SRE} is the SRE’s object misclassification rate (measured as held-out top-1 error), and ϵ_{exec} is the Action Decoder’s execution error given the correct object. Such linear horizon dependence on per-step classification error is standard in imitation-learning reduction analyses [27], [28]. The two terms are additive, not multiplicative, which means neither source compounds the other. Empirically, the ablation table (Table III) isolates each term: the drop from Unveiler to “No SRE” at 9–12 objects bounds ϵ_{SRE} ’s contribution, while the “No SRE” row itself upper-bounds ϵ_{exec} (8.6% partial / 7.8% full at 9–12 objects).

VI. EXPERIMENTS

We evaluate Unveiler in both simulated and real-world settings to assess effectiveness at retrieving occluded targets from cluttered scenes, answering: (i) How effectively can Unveiler retrieve targets across varying clutter levels? (ii) How does it compare to strong baselines? (iii) How well does it generalize? (iv) What is the impact of each component?

A. Baselines

We evaluate Unveiler against the following state-of-the-art baselines for object retrieval and manipulation in cluttered scenes: **Target-Conditioned PPG** [3], **Heur** (the heuristic policies used to train Unveiler), **ACT** [29], **CLIP-Grounding** [9], **GPT-4o** [5], **VILG** [8], and **ThinkGrasp** [10].

B. Evaluation Metrics

We evaluate our system using two complementary metrics that capture planning quality and execution robustness:

- **Task Completion Rate (%)**: The percentage of test episodes in which the robot successfully retrieves the target object.
- **Action Efficiency (Steps)**: The average number of actions taken, including obstacle removals and the final grasp.

C. Computational Efficiency Analysis

Unveiler achieves strong computational efficiency as shown in Table I. The reported ~ 260 ms for Unveiler is end-to-end and includes instance segmentation, crop extraction, SRE inference, and AD inference; baseline latencies similarly reflect their full perception-to-action pipelines where available. These results highlight how Unveiler maintains near real-time efficiency by separating spatial reasoning from action generation, avoiding end-to-end overhead and enabling deployment on resource-constrained robotic systems.

TABLE I
COMPUTATIONAL EFFICIENCY COMPARISON

Method	Params. (M)	Est. Inference (ms)	Efficiency ($\uparrow$)
Unveiler (Ours)	83.03	~ 260	1.0x
VILG	156.26	$\sim 29,300^a$	0.093–0.0047x
GPT-4o*	-	$> 6000^b$	$< 0.043x$
ThinkGrasp ⁺	-	$> 6000^b$	$< 0.043x$

*,⁺ Parameter count unavailable; requires cloud API inference.

^a Average shown; actual range 2807–55,741.9 ms.

^b Dependent on API latency; range is above 6000 ms.

D. Simulation Experiments

We validated the robustness of our system through testing on objects of varying scales from both YCB [30] and KIT [31] datasets, showing consistent performance across different object size distributions. We categorized the objects into two sets (seen and unseen sets) and used one for training (seen) and the other for evaluation (unseen). Each condition in Table II is evaluated over 30 episodes per method.

Our evaluation (as seen in Table II) demonstrates that Unveiler maintains superior performance across varying scene complexity while exhibiting more efficient planning compared to baseline approaches. We selected VILG [8] and ThinkGrasp [10] as baselines in addition to other approaches because they represent some of the most recent vision-based approaches for grasping in cluttered environments. VILG jointly integrates vision, language, and action without handcrafted grounding rules, making it highly relevant to our focus on relational reasoning. ThinkGrasp leverages GPT-4o for obstacle selection and part-level grasping to directly address sequential object removal, which aligns closely with the core challenge tackled by Unveiler. The results reveal three critical insights: first, Unveiler consistently requires fewer planning steps across all conditions, averaging 1.17–3.71 steps compared to baselines that often exceed 4+ steps in complex scenarios. This efficiency advantage is particularly pronounced in cluttered environments where methods like PPG and VILG require significantly more steps due to suboptimal sequencing decisions. Second, while all methods experience performance drops with increased clutter and occlusion, Unveiler demonstrates superior robustness, maintaining 53.8% success rate in the most challenging scenarios (9–12 objects with full occlusion) while most baselines drop below 35%, with some failing entirely (ACT and PPG show no completions). Third, the results expose method-specific limitations: GPT-4o exhibits planning inefficiency due to misalignment between its predictions and optimal removal sequences, requiring up to 4.22 steps in dense scenes, while the heuristic baseline fails dramatically as scene complexity increases, despite reasonable completion rates in low-clutter conditions. These results validate Unveiler’s core contribution: effective reasoning about obstacle removal sequences that translates to both higher task completion rates and more efficient planning across the full spectrum of scene complexity.

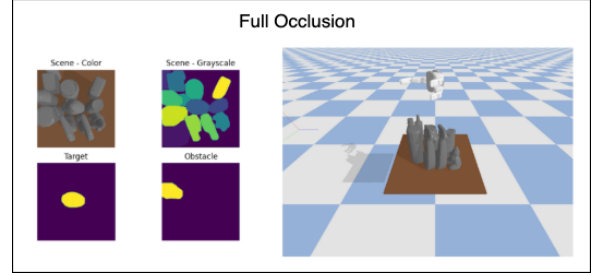
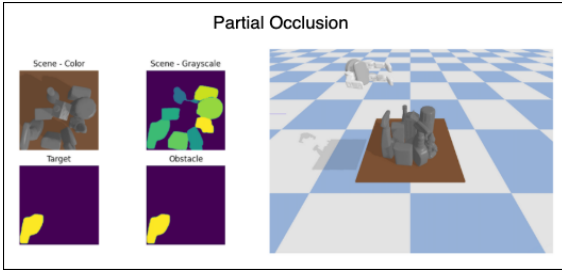


Fig. 3. Example scenes illustrating occlusion levels of the target object. (Left) The target object is partially occluded, with some visible surface area. (Right) The target object is fully occluded, entirely covered by surrounding objects, and not directly visible in the input view.

TABLE II

PERFORMANCE COMPARISON OF GPT-4o, CLIP-GROUNDING, VILG, THINKGRASP, ACT, PPG, HEUR, AND UNVEILER (OURS) ACROSS DIFFERENT CLUTTER DENSITIES AND OCCLUSION LEVELS. METRICS: TASK COMPLETION (%) AND AVERAGE PLANNING STEPS.

Clut.	Occl.	GPT-4o		CLIP-Ground.		VILG		ThinkGrasp		ACT		PPG		Heur		Unveiler (Ours)	
		%	Steps	%	Steps	%	Steps	%	Steps	%	Steps	%	Steps	%	Steps	%	Steps
2-6	Partial	86.0	1.50	80.5	1.67	75.0	3.0	73.3	3.00	45.2	1.14	40.6	4.42	87.3	2.35	96.1	1.32
2-6	Full	66.7	2.50	60.3	2.11	50.0	2.86	53.3	2.25	40.0	1.00	50.3	4.27	67.5	2.30	89.3	1.87
6-9	Partial	80.0	1.81	67.2	3.10	80.0	3.17	66.7	3.29	40.0	1.00	7.5	6.00	67.5	2.55	97.6	1.43
6-9	Full	60.0	4.22	66.7	4.20	40.0	3.33	60.0	3.78	10.0	1.00	4.4	3.00	47.8	3.43	90.0	3.31
9-12	Partial	66.7	3.10	53.0	4.38	46.7	2.86	46.7	3.42	25.7	1.00	4.4	4.00	50.0	3.40	92.6	2.86
9-12	Full	26.7	3.00	20.0	4.67	33.3	3.4	33.3	3.56	-	-	-	-	20.0	3.17	53.8	3.71

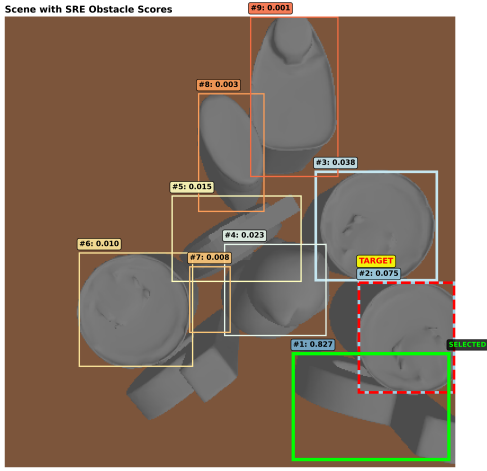


Fig. 4. Scene overlay with selection probabilities. Target outlined in red, selected obstacle in green.

E. Spatial Reasoning and Interpretability Analysis

The selection probability visualization in Figure 4 demonstrates that the SRE learns to identify optimal obstacles based on spatial relationships rather than simple proximity. The model consistently avoids selecting objects that are difficult to grasp due to workspace boundaries or neighboring objects, and shows sensitivity to cascading effects where removal might destabilize the scene. The interpretable selection probabilities provide transparency into the decision-making process, which is crucial for applications where understanding the robot’s reasoning is essential.

F. Ablation Studies

To understand the contribution of the individual components of Unveiler, we conducted a series of ablation studies, and the results are shown in Table III.

TABLE III

ABLATION STUDY. ARCHITECTURAL VARIANTS EVALUATED AT 6-9 OBJECT DENSITY; DENSITY GENERALIZATION SHOWN FOR NO SRE VS. FULL UNVEILER. METRICS: TASK COMPLETION (%) AND AVERAGE PLANNING STEPS.

Method	Density	Partial		Full	
		%	Steps	%	Steps
No SRE	6-9	96.0	2.10	76.0	3.39
Multi-Obstacle SRE	6-9	73.3	3.44	60.0	4.90
Binary Mask Input	6-9	56.7	2.94	36.7	4.55
Unveiler (Ours)	6-9	97.6	1.43	90.0	3.31
No SRE	9-12	84.0	2.91	46.0	6.39
SRE w/o RL finetuning	9-12	88.4	2.64	52.0	4.11
Unveiler (Ours)	9-12	92.6	2.86	53.8	3.71

- **SRE w/o RL finetuning:** Using only SRE without RL finetuning, the system underperforms relative to the full Unveiler system by **3.3%**, confirming that RL discovers strategies that the heuristic supervisor cannot provide.
- **No SRE:** Removing the spatial relationship encoder and defaulting to random grasps reduces completion by **13.6%** under full occlusion while slightly improving partial occlusion performance by **3.4%**, demonstrating its critical role in complex scenarios.

- **Multi-Obstacle Selection:** Predicting all removable obstacles in one shot degrades performance by **25.2%** (partial) and **33.3%** (full occlusion), demonstrating the importance of sequential reasoning.
- **Binary Mask Input:** Replacing rich object crops with binary masks causes significant drops of **42.1%** (partial) and **53.3%** (full occlusion), validating the importance of detailed visual representations.

G. Demonstration on a Real Robot

We validate our system on a Dofbot-Pro manipulator (as shown in Figure 1) equipped with a parallel gripper. The perception system consists of a calibrated Intel RealSense camera mounted at the top of the robot base, providing top-down RGB-D data at 480×640 resolution.

The architecture’s modular design facilitates direct sim-to-real transfer with minimal adaptation. The SRE transfers zero-shot: first, the top-down camera geometry used in deployment matches the simulator, preserving spatial scale and centroid coordinates. Second, the Action Decoder’s rotation-invariant prediction aggregates over 16 orientations, making grasps robust to small pose deviations between the simulated and physical object placements. The heightmap for the Action Decoder was generated directly from the single top-down Intel RealSense camera mounted on the robot base. While this provides a less complete 3D point cloud than the dual-camera setup used in simulation (which mitigates self-occlusion from the robot’s arm), our policy outperformed other approaches in real-world trials as shown in Table IV. The only real-world adaptation was a one-time geometric recalibration of the workspace bounds to the physical table extent; no learned component was retrained or fine-tuned. Critically, arm kinematics are encoded solely in these non-learned bounds and not in the policy itself, so the shift from the Barrett hand to the Dofbot-Pro introduces no domain gap for the SRE. This confirms that the domain gap between simulated and physical heightmaps does not significantly degrade performance.

H. Scene Reasoning Evaluation on Real Images

TABLE IV
OBJECT SELECTION ACCURACY ON REAL SCENES ($N = 35$)

Method	2 - 4 (%)	5 - 8 (%)
CLIP-Grounding	65	37
GPT-4o	40	26
SRE (Ours)	60	54

To isolate the SRE’s spatial reasoning from execution noise, we evaluate object-selection accuracy directly on 35 real top-down tabletop scenes. This decoupled evaluation is natural given the factored architecture: the SRE’s discrete output can be assessed independently of any physical arm.

Results are shown in Table IV. In sparse scenes (2–4 objects), CLIP-Grounding performs comparably, as appearance similarity suffices when occlusion is limited. In denser scenes (5–8 objects), the SRE (54%) substantially outperforms

CLIP-Grounding (37%) and GPT-4o (26%), confirming that geometric reasoning about occlusion and periphery, not appearance similarity, drives the SRE’s advantage in clutter. These results confirm that the learned spatial reasoning transfers to real scenes without any fine-tuning.

VII. CONCLUSION

We have presented Unveiler, a compositionally decomposed framework for sequential object retrieval in dense clutter. The system separates long-horizon spatial reasoning (SRE) from low-level action execution (Action Decoder) via a clean discrete interface, enabling each module to specialize, be trained independently, and be transferred independently. This modularity is not merely an implementation choice: it is the central design principle, and our experiments demonstrate its tangible benefits.

Modularity as a design principle. Four properties of Unveiler follow directly from its modular decomposition and would be difficult or impossible to achieve with a monolithic policy:

- 1) *Independent trainability:* The SRE and Action Decoder are trained on separate objectives with no shared gradient flow, allowing each to use the most appropriate training signal (IL+RL for the SRE; supervised learning for the Action Decoder).
- 2) *Modular transfer:* The SRE transfers zero-shot from simulation to real top-down images (Table IV) because the module boundary (a discrete object index) isolates it from arm kinematics and hardware variation, while the Action Decoder requires only a one-time geometric recalibration of workspace bounds—no learned weights are retrained or fine-tuned.
- 3) *Independent failure diagnosis:* The interpretable selection probabilities allow inspection of which object the SRE prioritized and why, and the error bound (Section V) cleanly separates SRE misclassification error from Action Decoder execution error.
- 4) *Composability:* The SRE’s discrete output is a natural interface for composition with other systems. It can serve as a specialized spatial reasoning head for a VLA, as a demonstration generator for cluttered-scene datasets, or as a drop-in planning module when the Action Decoder is replaced with a richer manipulation primitive.

Limitations. The module interface relies on instance segmentation, which remains a source of error in very dense clutter. The push-grasp Action Decoder is deliberately simple; replacing it with a more dexterous primitive without retraining the SRE is a direct extension enabled by the modular design. The demonstrated effectiveness of explicit compositional decomposition suggests continued relevance for purpose-built modular architectures alongside large-scale models.

REFERENCES

- [1] B. Tang, M. Corsaro, G. Konidaris, S. Nikolaidis, and S. Tellex, “Learning collaborative pushing and grasping policies in dense clut-

- ter,” in *2021 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2021, pp. 6177–6184.
- [2] A. Zeng, P. Florence, J. Tompson, S. Welker, J. Chien, M. Attarian, T. Armstrong, I. Krasin, D. Duong, V. Sindhwani *et al.*, “Transporter networks: Rearranging the visual world for robotic manipulation,” in *Conference on Robot Learning*. PMLR, 2021, pp. 726–747.
 - [3] M. Kiatos, I. Sarantopoulos, L. Koutras, S. Malassiotis, and Z. Doulgeri, “Learning push-grasping in dense clutter,” *IEEE Robotics and Automation Letters*, vol. 7, no. 4, pp. 8783–8790, 2022.
 - [4] D. Li, C. Zhao, S. Yang, R. Song, X. Li, and W. Zhang, “Mpgnet: Learning move-push-grasping synergy for target-oriented grasping in occluded scenes,” in *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2024, pp. 5064–5071.
 - [5] A. Hurst, A. Lerer, A. P. Goucher, A. Perelman, A. Ramesh, A. Clark, A. Ostrow, A. Welihinda, A. Hayes, A. Radford *et al.*, “Gpt-4o system card,” *arXiv preprint arXiv:2410.21276*, 2024.
 - [6] K. Black, N. Brown, D. Driess, A. Esmail, M. Equi, C. Finn, Y. Fusai, L. Groom, K. Hausman, B. Ichter *et al.*, “pi0: A vision-language-action flow model for general robot control,” *arXiv preprint arXiv:2410.24164*, 2024.
 - [7] J. Bjorck, F. Castañeda, N. Cherniadev, X. Da, R. Ding, L. Fan, Y. Fang, D. Fox, F. Hu, S. Huang *et al.*, “Gr00t n1: An open foundation model for generalist humanoid robots,” *arXiv preprint arXiv:2503.14734*, 2025.
 - [8] K. Xu, S. Zhao, Z. Zhou, Z. Li, H. Pi, Y. Zhu, Y. Wang, and R. Xiong, “A joint modeling of vision-language-action for target-oriented grasping in clutter,” pp. 11 597–11 604, 2023.
 - [9] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark *et al.*, “Learning transferable visual models from natural language supervision,” in *International conference on machine learning*. PMLR, 2021, pp. 8748–8763.
 - [10] Y. Qian, X. Zhu, O. Biza, S. Jiang, L. Zhao, H. Huang, Y. Qi, and R. Platt, “Thinkgrasp: A vision-language system for strategic part grasping in clutter,” *arXiv preprint arXiv:2407.11298*, 2024.
 - [11] E. Coumans and Y. Bai, “Pybullet, a python module for physics simulation for games, robotics and machine learning,” <http://pybullet.org>, 2016–2021.
 - [12] A. Zeng, S. Song, S. Welker, J. Lee, A. Rodriguez, and T. Funkhouser, “Learning synergies between pushing and grasping with self-supervised deep reinforcement learning,” in *2018 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2018, pp. 4238–4245.
 - [13] Y. Deng, X. Guo, Y. Wei, K. Lu, B. Fang, D. Guo, H. Liu, and F. Sun, “Deep reinforcement learning for robotic pushing and picking in cluttered environment,” in *2019 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. Ieee, 2019, pp. 619–626.
 - [14] Y. Yang, H. Liang, and C. Choi, “A deep learning approach to grasping the invisible,” *IEEE Robotics and Automation Letters*, vol. 5, no. 2, pp. 2232–2239, 2020.
 - [15] Y. Jiang, A. Gupta, Z. Zhang, G. Wang, Y. Dou, Y. Chen, L. Fei-Fei, A. Anandkumar, Y. Zhu, and L. Fan, “Vima: General robot manipulation with multimodal prompts,” *arXiv preprint arXiv:2210.03094*, vol. 2, no. 3, p. 6, 2022.
 - [16] Y. Zhu, A. Joshi, P. Stone, and Y. Zhu, “Viola: Imitation learning for vision-based manipulation with object proposal priors,” in *Conference on Robot Learning*. PMLR, 2023, pp. 1199–1210.
 - [17] Y. Zhu, Z. Jiang, P. Stone, and Y. Zhu, “Learning generalizable manipulation policies with object-centric 3d representations,” *arXiv preprint arXiv:2310.14386*, 2023.
 - [18] S. Ferraro, P. Mazzaglia, T. Verbelen, and B. Dhoedt, “Focus: object-centric world models for robotic manipulation,” *Frontiers in Neuro-robotics*, vol. 19, p. 1585386, 2025.
 - [19] L. Meng, Z. Qi, L. Shuchang, W. Chunlei, M. Yujing, C. Guangliang, and Y. Chenguang, “Ovgnet: A unified visual-linguistic framework for open-vocabulary robotic grasping,” *arXiv preprint arXiv:2407.13175*, 2024.
 - [20] C. Eze and C. Crick, “Learning by watching: A review of video-based learning approaches for robot manipulation,” *IEEE Access*, 2025.
 - [21] L. P. Kaelbling and T. Lozano-Pérez, “Hierarchical task and motion planning in the now,” *2011 IEEE International Conference on Robotics and Automation*, pp. 1470–1477, 2011.
 - [22] C. R. Garrett, R. Chitnis, R. Holladay, B. Kim, T. Silver, L. P. Kaelbling, and T. Lozano-Pérez, “Integrated task and motion planning,” *Annual review of control, robotics, and autonomous systems*, vol. 4, pp. 265–293, 2021.
 - [23] A. Vaswani, “Attention is all you need,” *Advances in Neural Information Processing Systems*, 2017.
 - [24] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016, pp. 770–778.
 - [25] O. Russakovsky, J. Deng, H. Su, J. Krause, S. Satheesh, S. Ma, Z. Huang, A. Karpathy, A. Khosla, M. Bernstein *et al.*, “Imagenet large scale visual recognition challenge,” *International journal of computer vision*, vol. 115, pp. 211–252, 2015.
 - [26] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal policy optimization algorithms,” *arXiv preprint arXiv:1707.06347*, 2017.
 - [27] S. Ross and J. A. Bagnell, “Efficient reductions for imitation learning,” in *Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics (AISTATS)*, 2010.
 - [28] S. Ross, G. Gordon, and J. A. Bagnell, “A reduction of imitation learning and structured prediction to no-regret online learning,” in *Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics (AISTATS)*, 2011.
 - [29] T. Z. Zhao, V. Kumar, S. Levine, and C. Finn, “Learning fine-grained bimanual manipulation with low-cost hardware,” *arXiv preprint arXiv:2304.13705*, 2023.
 - [30] B. Calli, A. Singh, A. Walsman, S. Srinivasa, P. Abbeel, and A. M. Dollar, “The ycb object and model set: Towards common benchmarks for manipulation research,” in *2015 international conference on advanced robotics (ICAR)*. IEEE, 2015, pp. 510–517.
 - [31] A. Kasper, Z. Xue, and R. Dillmann, “The kit object models database: An object model database for object recognition, localization and manipulation in service robotics,” *The International Journal of Robotics Research*, vol. 31, no. 8, pp. 927–934, 2012.